

LISTA DOBLEMENTE ENLAZADA

```
import java.util.LinkedList;

class Alumno {
    private String nombre;
    private int numeroControl;

    public Alumno(String nombre, int numeroControl) {
        this.nombre = nombre;
        this.numeroControl = numeroControl;
    }

    public String getNombre() {
        return nombre;
    }

    public int getNumeroControl() {
        return numeroControl;
    }

    public String toString() {
        return "Nombre: " + nombre + ", Número de Control: " + numeroControl;
    }
}

public class EjemploListaDobleEnlazada {
    public static void main(String[] args) {
        // Crear una lista doblemente enlazada de objetos Alumno
        LinkedList<Alumno> listaAlumnos = new LinkedList<>();

        // Agregar alumnos a la lista
        listaAlumnos.add(new Alumno("Ana", 123456));
        listaAlumnos.add(new Alumno("Luis", 234567));
        listaAlumnos.add(new Alumno("Pedro", 345678));
        listaAlumnos.add(new Alumno("Maria", 456789));

        // Imprimir los alumnos de la lista
        System.out.println("Lista de alumnos:");
        for (Alumno alumno : listaAlumnos) {
            System.out.println(alumno);
        }

        // Acceder al primer y último elemento
        System.out.println("\nPrimer alumno de la lista: " + listaAlumnos.getFirst());
        System.out.println("Último alumno de la lista: " + listaAlumnos.getLast());

        // Modificar un alumno
        listaAlumnos.set(1, new Alumno("Carlos", 234567));
        System.out.println("\nLista de alumnos después de la modificación:");
        for (Alumno alumno : listaAlumnos) {
            System.out.println(alumno);
        }

        // Eliminar un alumno de la lista
        listaAlumnos.remove(2); // Elimina al alumno en el índice 2
        System.out.println("\nLista de alumnos después de la eliminación:");
        for (Alumno alumno : listaAlumnos) {
            System.out.println(alumno);
        }

        // Tamaño de la lista
        System.out.println("\nTamaño de la lista: " + listaAlumnos.size());
    }
}
```

En Java, las **listas doblemente enlazadas** se pueden implementar mediante una clase personalizada, dado que Java no tiene punteros explícitos como C++ (**se usan referencias de objetos**). Aquí tienes un ejemplo básico de una lista doblemente enlazada en Java, donde cada nodo apunta a su nodo anterior y su nodo siguiente.

```
class Nodo {
    int dato;
    Nodo siguiente;
    Nodo anterior;

    public Nodo(int dato) {
        this.dato = dato;
        this.siguiente = null;
        this.anterior = null;
    }
}

class ListaDoble {
    private Nodo cabeza;
    private Nodo cola;

    public ListaDoble() {
        this.cabeza = null;
        this.colas = null;
    }

    // Agregar al final de la lista
    public void agregar(int valor) {
        Nodo nuevo = new Nodo(valor);

        if (cabeza == null) {
            cabeza = cola = nuevo;
        } else {
            cola.siguiente = nuevo;
            nuevo.anterior = cola;
            cola = nuevo;
        }
    }

    // Eliminar un nodo específico
    public void eliminar(int valor) {
        Nodo actual = cabeza;

        while (actual != null) {
            if (actual.dato == valor) {
                if (actual.anterior != null) {
                    actual.anterior.siguiente = actual.siguiente;
                } else {
                    cabeza = actual.siguiente;
                }

                if (actual.siguiente != null) {
                    actual.siguiente.anterior = actual.anterior;
                } else {
                    cola = actual.anterior;
                }
            }
            actual = actual.siguiente;
        }

        System.out.println("Nodo con valor " + valor + " eliminado.");
        return;
    }
}

// Imprimir la lista de inicio a fin
public void imprimirAdelante() {
    Nodo actual = cabeza;
    System.out.print("Lista en orden: ");
    while (actual != null) {
        System.out.print(actual.dato + " ");
        actual = actual.siguiente;
    }
    System.out.println();
}

// Imprimir la lista de fin a inicio
public void imprimirAtras() {
    Nodo actual = cola;
    System.out.print("Lista en orden inverso: ");
    while (actual != null) {
        System.out.print(actual.dato + " ");
        actual = actual.anterior;
    }
    System.out.println();
}

public class Main {
    public static void main(String[] args) {
        ListaDoble lista = new ListaDoble();

        lista.agregar(10);
        lista.agregar(20);
        lista.agregar(30);

        lista.imprimirAdelante(); // Muestra: 10 20 30
        lista.imprimirAtras();   // Muestra: 30 20 10

        lista.eliminar(20);

        lista.imprimirAdelante(); // Muestra: 10 30
        lista.imprimirAtras();   // Muestra: 30 10
    }
}
```

LISTA DOBLEMENTE ENLAZADA reproductor

```
class Cancion {
    String titulo;
    Cancion siguiente;
    Cancion anterior;

    public Cancion(String titulo) {
        this.titulo = titulo;
        this.siguiente = null;
        this.anterior = null;
    }
}

class ListaReproduccion {
    private Cancion primera;
    private Cancion actual;

    // Agregar una canción al final de la lista
    public void agregarCancion(String titulo) {
        Cancion nuevaCancion = new Cancion(titulo);

        if (primera == null) {
            primera = nuevaCancion;
            actual = primera;
        } else {
            Cancion ultima = primera;
            while (ultima.siguiente != null) {
                ultima = ultima.siguiente;
            }
            ultima.siguiente = nuevaCancion;
            nuevaCancion.anterior = ultima;
        }
    }

    // Reproducir la canción actual
    public void reproducirActual() {
        if (actual != null) {
            System.out.println("Reproduciendo: " +
                actual.titulo);
        } else {
            System.out.println("No hay canciones en la lista de
                reproducción.");
        }
    }

    // Ir a la siguiente canción
    public void siguienteCancion() {
        if (actual != null && actual.siguiente != null) {
            actual = actual.siguiente;
            reproducirActual();
        } else {
            System.out.println("No hay más canciones hacia
                adelante.");
        }
    }

    // Ir a la canción anterior
    public void cancionAnterior() {
        if (actual != null && actual.anterior != null) {
            actual = actual.anterior;
            reproducirActual();
        } else {
            System.out.println("No hay más canciones hacia
                atrás.");
        }
    }
}

public class ReproductorMusica {
    public static void main(String[] args) {
        ListaReproduccion listaReproduccion = new
            ListaReproduccion();

        // Agregar canciones a la lista de reproducción
        listaReproduccion.agregarCancion("Canción 1 - Intro");
        listaReproduccion.agregarCancion("Canción 2 - Éxito");
        listaReproduccion.agregarCancion("Canción 3 -
            Clásico");
        listaReproduccion.agregarCancion("Canción 4 - Final");

        // Reproducir la lista de reproducción
        listaReproduccion.reproducirActual(); // Reproduce la
            primera canción

        // Navegar entre las canciones
        listaReproduccion.siguienteCancion(); // Reproduce la
            segunda canción
        listaReproduccion.siguienteCancion(); // Reproduce la
            tercera canción
        listaReproduccion.cancionAnterior(); // Regresa a la
            segunda canción
        listaReproduccion.siguienteCancion(); // Reproduce la
            tercera canción
        listaReproduccion.siguienteCancion(); // Reproduce la
            cuarta canción
        listaReproduccion.siguienteCancion(); // No hay más
            canciones hacia adelante
    }
}
```

VENTAJAS Y DESVENTAJAS DE LAS LISTAS SIMPLES Y LAS LISTAS DOBLEMENTE ENLAZADAS

Ventajas	Listas Simplemente Enlazadas	Ventajas	Listas Doblemente Enlazadas
Menor uso de memoria	Cada nodo solo almacena una referencia al siguiente nodo, lo que reduce el consumo de memoria en comparación con una lista doblemente enlazada.	Acceso bidireccional	Los nodos contienen punteros tanto al nodo siguiente como al anterior, lo que permite recorrer la lista en ambas direcciones. Esto es útil para aplicaciones que requieren moverse en ambas direcciones, como en un reproductor de música o una lista de historial.
Estructura más sencilla	Las listas simplemente enlazadas tienen una estructura de nodos más simple, lo cual facilita su implementación y manejo en operaciones básicas.	Eliminación y modificación más sencilla	Al tener un puntero al nodo anterior, se pueden eliminar nodos intermedios con más facilidad, sin necesidad de recorrer la lista desde el inicio.
Inserción eficiente al inicio	Agregar un nodo al comienzo de la lista es rápido y solo requiere ajustar un puntero.	Mayor flexibilidad en operaciones de lista	Las listas doblemente enlazadas permiten realizar operaciones complejas, como agregar, eliminar o mover nodos de forma más eficiente en algunos casos.
Adecuadas para aplicaciones ligeras	Son ideales para aplicaciones donde solo se requiere acceso secuencial y no se necesita retroceder en la lista.		
Desventajas			
Acceso unidireccional	Solo se puede recorrer la lista en una dirección (de inicio a fin), lo cual limita su funcionalidad en aplicaciones que requieren recorrer la lista en ambas direcciones	Mayor uso de memoria	Cada nodo almacena dos punteros (anterior y siguiente), lo que aumenta el consumo de memoria en comparación con las listas simplemente enlazadas.
Eliminación más lenta de nodos	Para eliminar un nodo intermedio, se debe recorrer la lista desde el inicio para encontrar el nodo previo, lo que puede ser ineficiente.	Estructura más compleja	La implementación es más compleja debido a los punteros adicionales, lo que requiere más atención al gestionar la inserción y eliminación de nodos.
Complejidad en operaciones de retroceso	No se puede acceder directamente al nodo anterior, lo que limita la manipulación bidireccional.	Mayor costo de mantenimiento de punteros:	En cada inserción o eliminación de nodos, deben actualizarse dos punteros (anterior y siguiente), lo que puede incrementar el tiempo de ejecución en comparación con listas simples en aplicaciones de gran escala.

Resumen

Usa listas simplemente enlazadas cuando necesitas una estructura ligera, con inserciones y eliminaciones rápidas al inicio o cuando solo necesitas recorrido en una dirección.

Usa listas doblemente enlazadas si tu aplicación requiere recorrido en ambas direcciones, y si las operaciones de inserción y eliminación en cualquier parte de la lista son frecuentes.